



Habib University
shaping futures

CS201 – Data Structures II

Lecture 10

Dr. Muhammad Mobeen Movania
(Content has been taken from lecture slides of
Dr. Faisal Alvi)

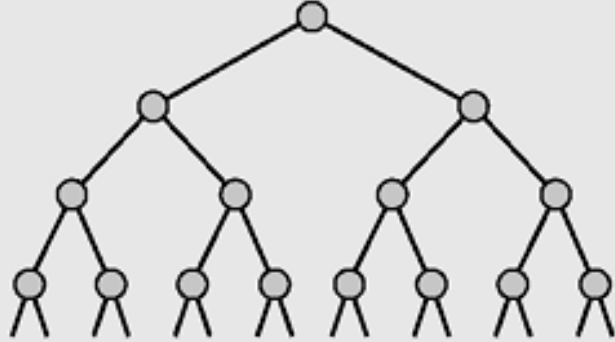


Outline

- Binary tree and height relationship
- Balancing of a Binary Tree
- Introduction to AVL Trees
- Operations
 - Rotation
 - Insertion
 - Deletion

Relationship between no. of nodes and height in a binary tree



	<i>Height</i>	<i>Nodes at One Level</i>	<i>Nodes at All Levels</i>
	1	$2^0 = 1$	$1 = 2^1 - 1$
	2	$2^1 = 2$	$3 = 2^2 - 1$
	3	$2^2 = 4$	$7 = 2^3 - 1$
	4	$2^3 = 8$	$15 = 2^4 - 1$
	⋮		
	11	$2^{10} = 1,024$	$2,047 = 2^{11} - 1$
	⋮		
	14	$2^{13} = 8,192$	$16,383 = 2^{14} - 1$
	⋮		
	h	2^{h-1}	$n = 2^h - 1$
	⋮		



Why is it important to have a balanced binary tree?

- Recall from the last session that the height of a binary search tree with n nodes can vary from $\log n$ (balanced) to n (degenerate).
- Since the complexity of find, insert and delete operations depends on the height, it is important to maintain the height of the tree to $\log n$.
- A binary tree is **height-balanced** or **balanced** if the difference in height of both subtrees of any node in the tree is either zero or one

How do you balance a BST?

- One way to construct a balanced BST is to sort the tree data first.
- Then, each element can be inserted in the order in which elements in binary search are located as follows:

```
void balance (T data[], int first, int last) {  
    if (first <= last) {  
        int middle = (first + last)/2;  
        insert(data[middle]); // into the BST  
        balance(data, first, middle-1);  
        balance(data, middle+1, last);  
    }  
}
```

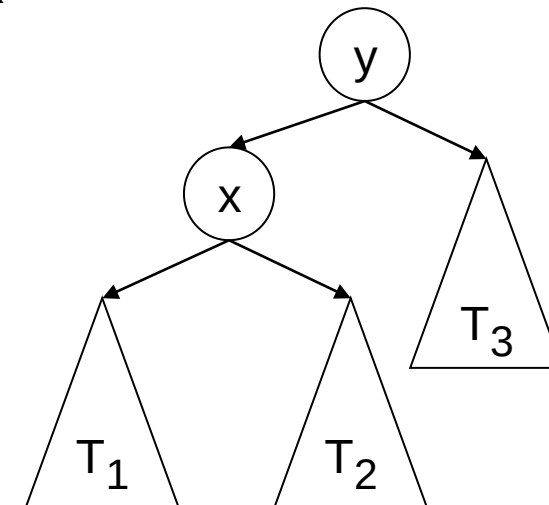
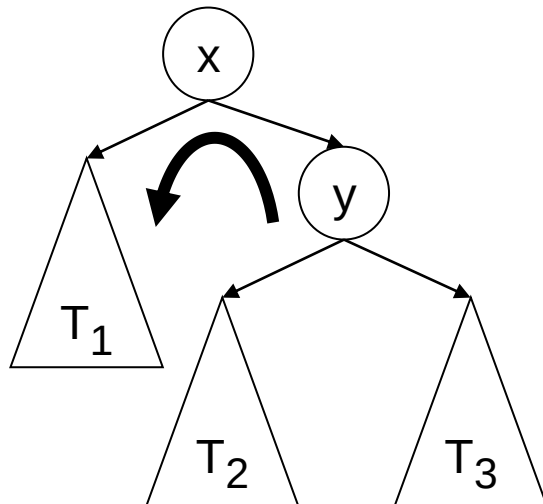
Rotations

- Another way to balance a BST is rotations
- A rotation is a process of switching children and parents among two or three adjacent nodes.
- There are four different types of rotations in order to balance a BST:
 - Single Left Rotation
 - Single Right Rotation
 - Double Left Right Rotation
 - Double Right Left Rotation

Single Left Rotation

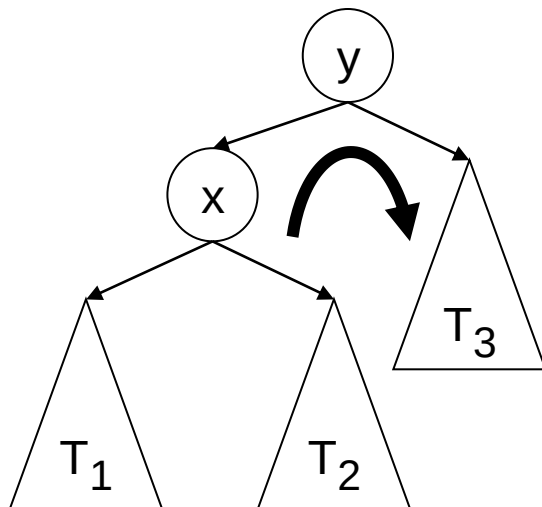
- Single left rotation:
 - The right child y of a node x becomes x 's parent.
 - x becomes the left child of y .
 - The left subtree T_2 of y , if any, becomes the right child of x .

a left rotation of y about x

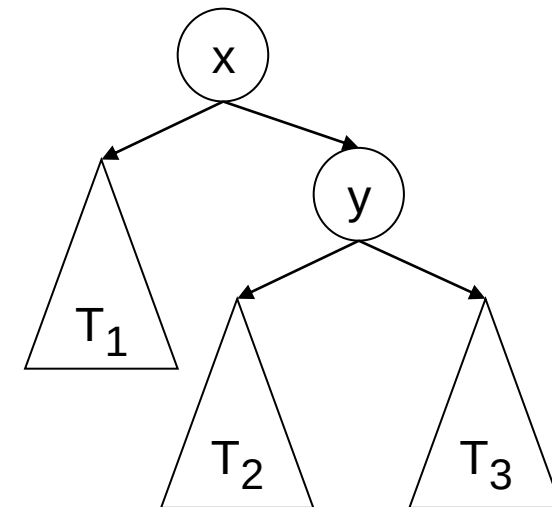


Single Right Rotation

- Single right rotation:
 - The left child x of a node y becomes y 's parent.
 - y becomes the right child of x .
 - The right subtree T_2 of x , if any, becomes the left child of y .

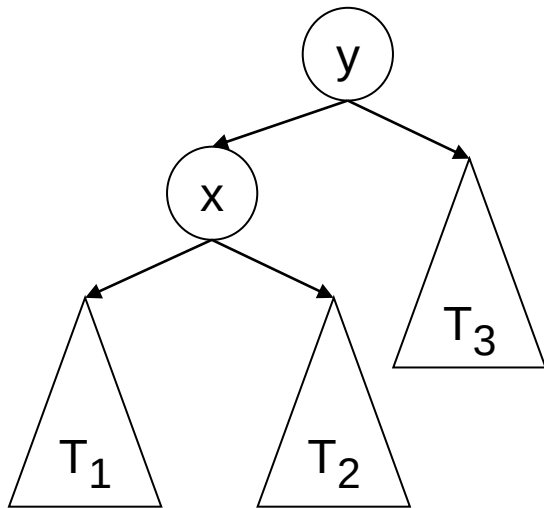


a right rotation of x about y

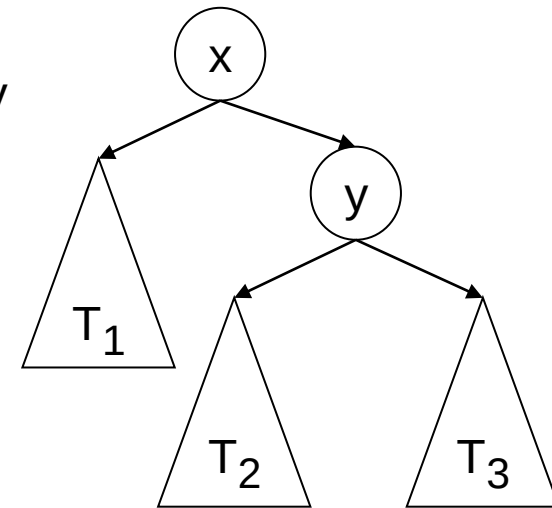


BST Order Property After a Rotation

- A rotation does not affect the ordering property of a BST (Binary Search Tree).



a right or left rotation of x about y



- BST ordering property requirement: BST ordering property requirement:

$$T_1 < x < y$$

$$x < T_2 < y$$

$$x < y < T_3$$

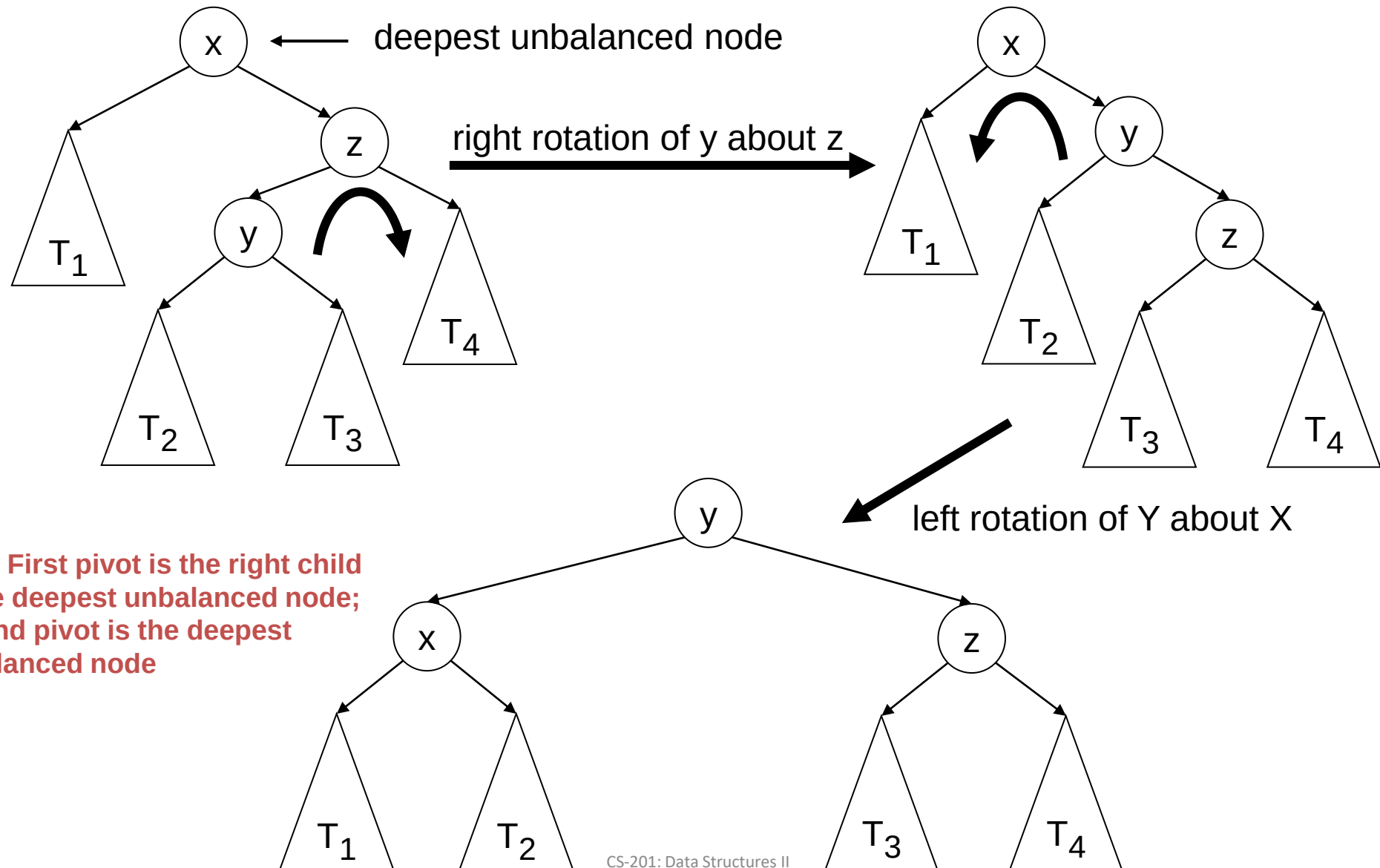
same as

$$T_1 < x < y$$

$$x < T_2 < y$$

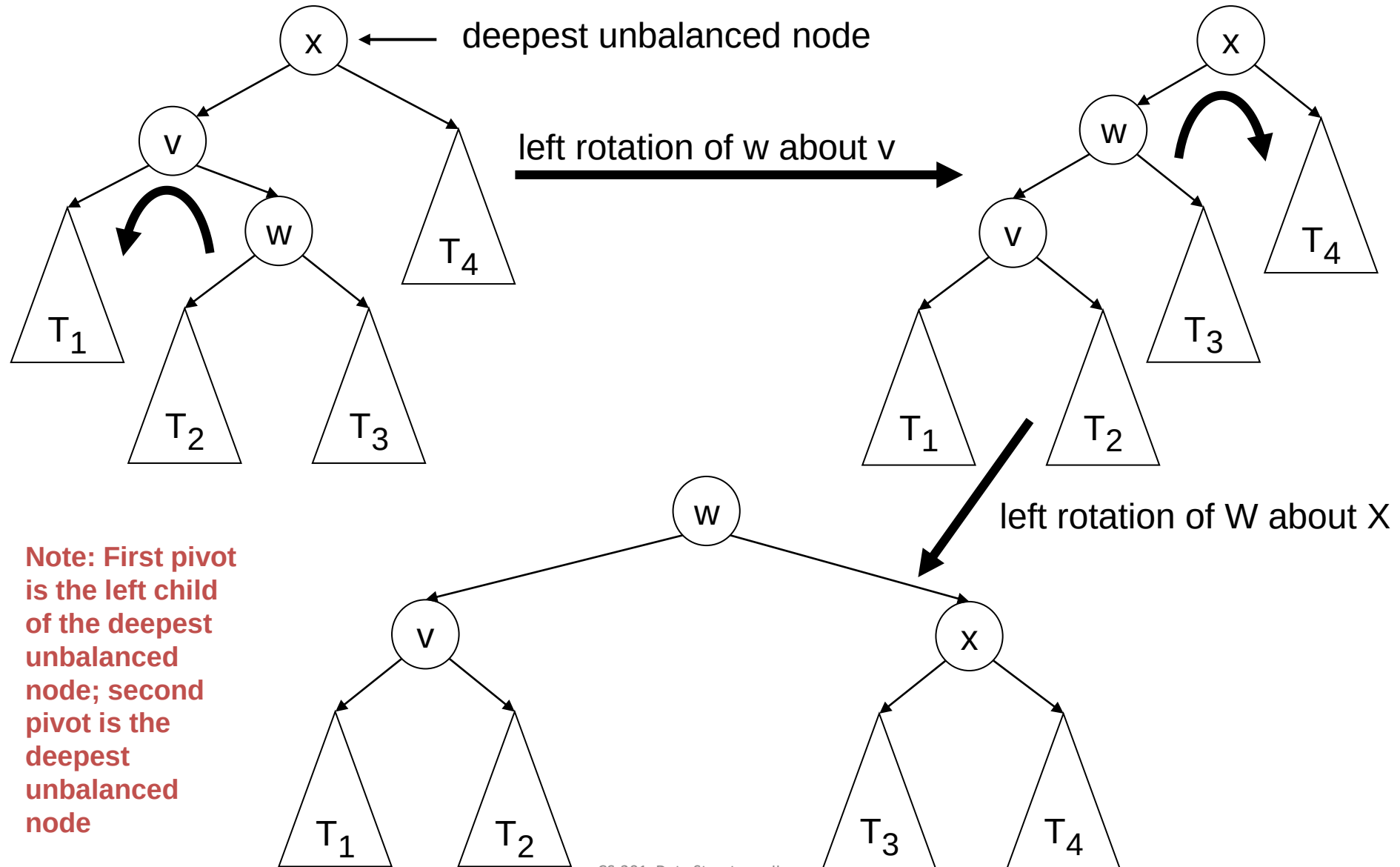
$$x < y < T_3$$

Double Right-Left Rotation



Note: First pivot is the right child of the deepest unbalanced node; second pivot is the deepest unbalanced node

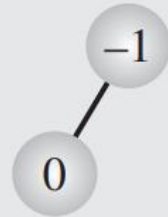
Double Left-Right Rotation



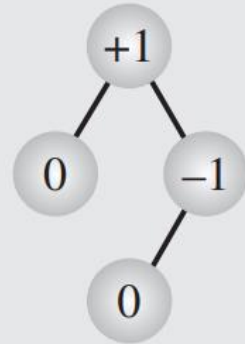
AVL Trees

- A balanced binary tree in which the height of the left and right subtrees of **every node** differ by at most one.
 - Balanced factor = $H_{right} - H_{left}$
- For an AVL tree, balance factors of every node should be +1, 0, or -1.
 - Every subtree in AVL Tree is also an AVL Tree
- If the balance factor of any node is >1 or <-1, we must balance it to make the balance factor between +1,0,-1
 - We do one or more rotation operations to do this balancing as we will see later
- An AVL tree is rebalanced after each insertion or deletion.
 - The height-balance property ensures that the height of an AVL tree with n nodes is $O(\log n)$.
 - Searching, insertion, and deletion are all $O(\log n)$.

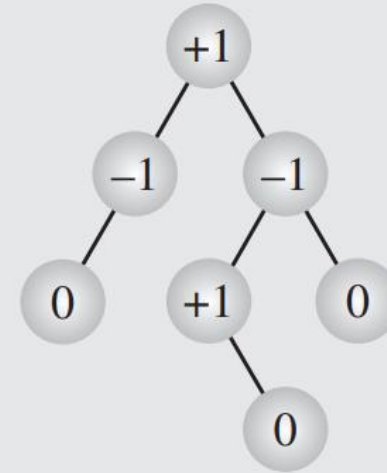
Example of AVL Trees



(a)



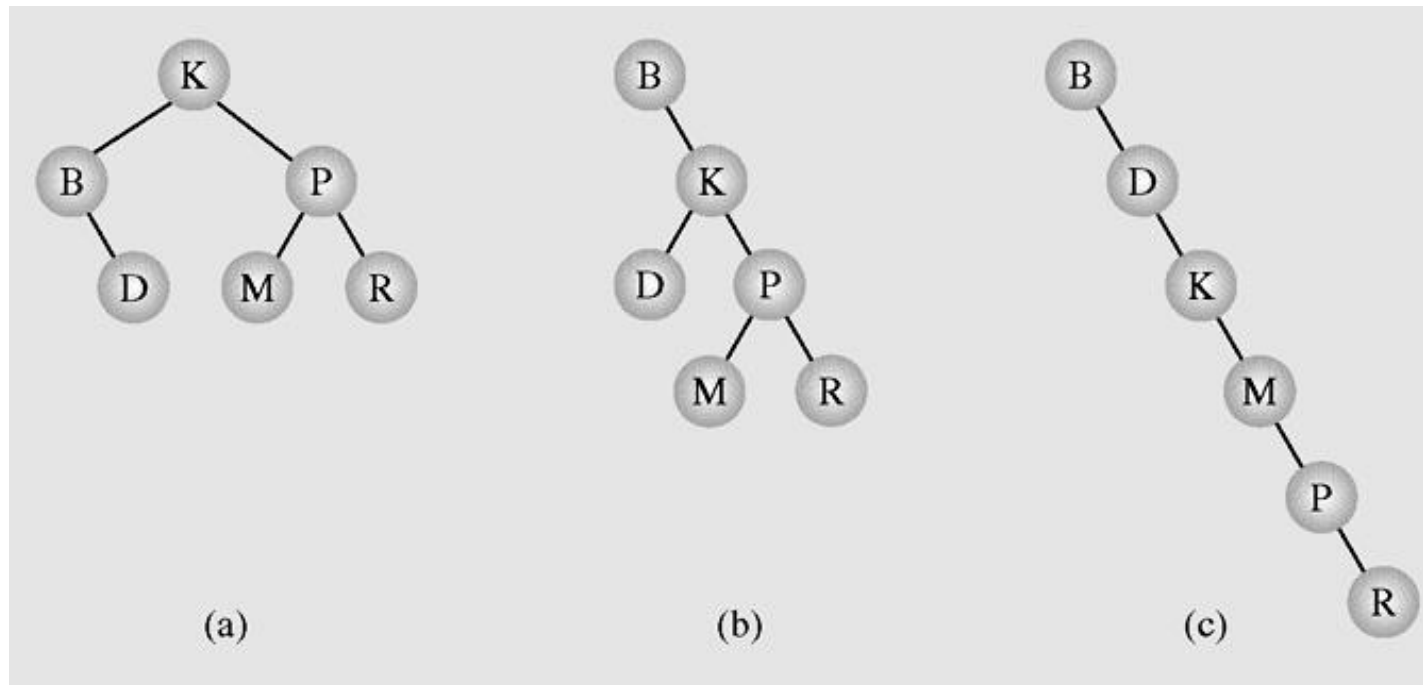
(b)



(c)

Exercise

- Calculate balance factors of each nodes for the following BSTs? Also share if they are AVL trees?
 - Balance factor = $H_{right} - H_{left}$

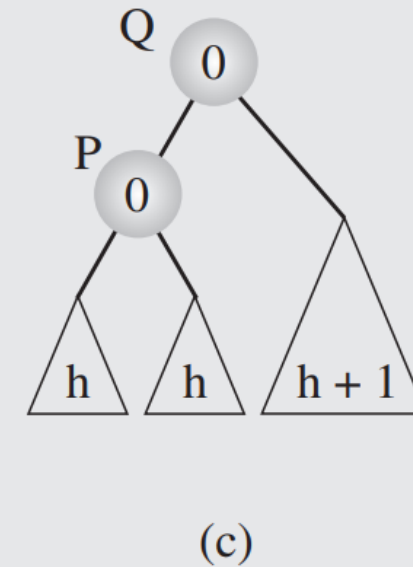
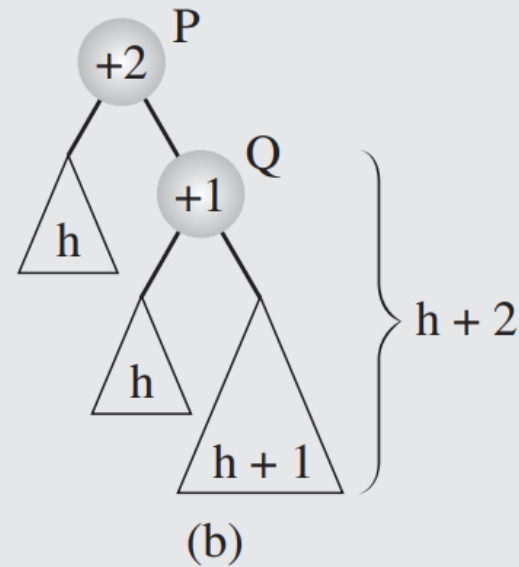
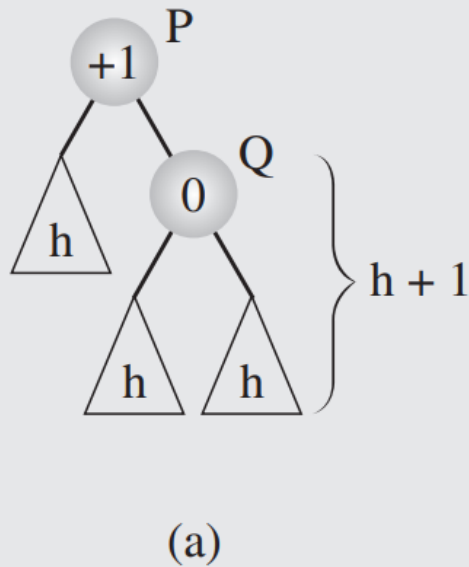


Operations in AVL Trees

- Insertion or deletion in an ordinary Binary Search Tree can cause large imbalances.
- In the worst case searching an imbalanced Binary Search Tree is $O(n)$ if the BST is left or right skewed (see previous figure sub-image (c))
- An AVL tree is rebalanced after each insertion or deletion.
 - The height-balance property ensures that the height of an AVL tree with n nodes is $O(\log n)$.
 - Searching, insertion, and deletion are all $O(\log n)$.

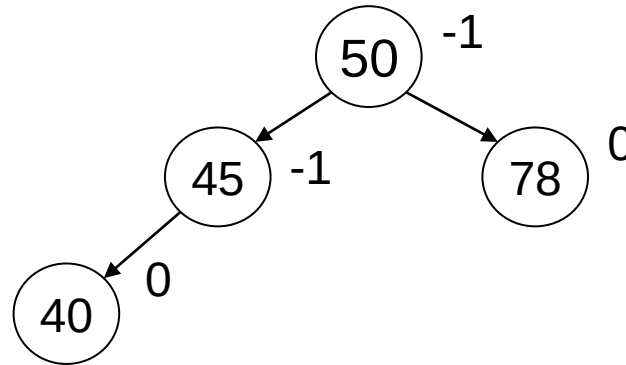
Rebalancing of an AVL Tree by rotation

FIGURE 6.41 Balancing a tree after insertion of a node in the right subtree of node Q.



Example of insertion in AVL tree

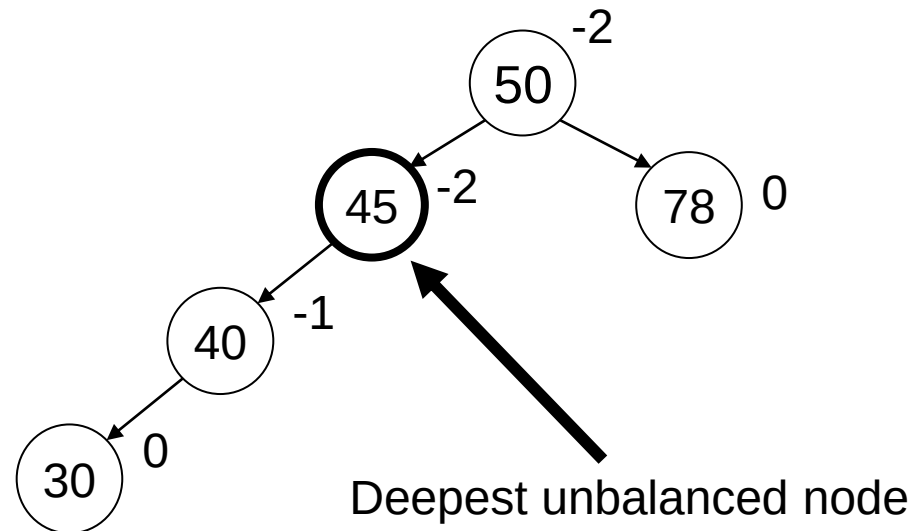
- Consider the following AVL Tree with values contained in the node and balance factors written on the top right.



- We insert a new value 30 into this binary tree

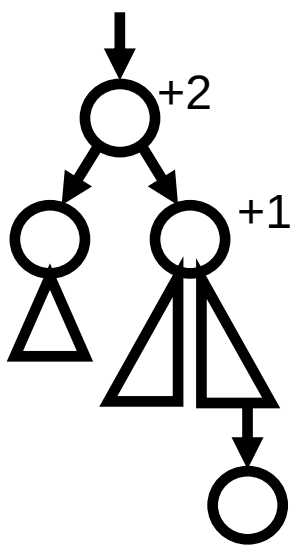
Example of insertion in AVL tree

- Insert(30) makes the AVL tree unbalanced
 - due to the deepest node, which is an ancestor of a deleted or an inserted node, and whose balance factor has changed to -2 or $+2$ requires rotation to rebalance the tree.

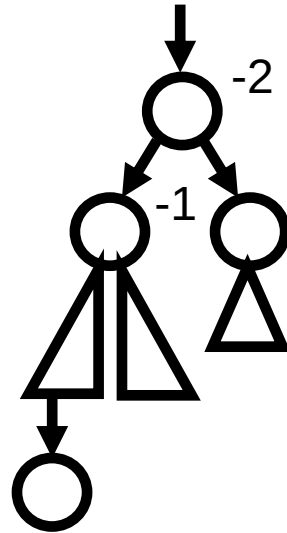


Balancing an AVL Tree

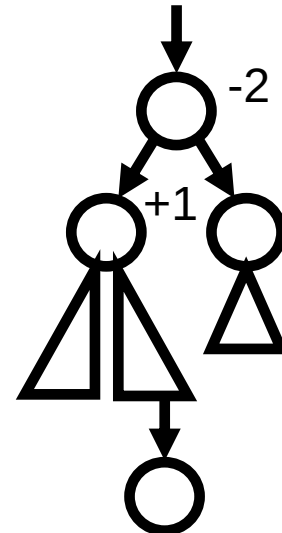
- An unbalanced AVL may be rebalanced by using the following four rotations
 - Case 1(a): Left rotation (similar to the left rotation in BST shown earlier)
 - Case 1(b): Right rotation (similar to the right rotation in BST shown earlier)
 - Case 2(a): A double left-right rotation (that is a left rotation followed by a right rotation).
 - Case 2(b): A double right-left rotation (that is a right rotation followed by a left rotation).



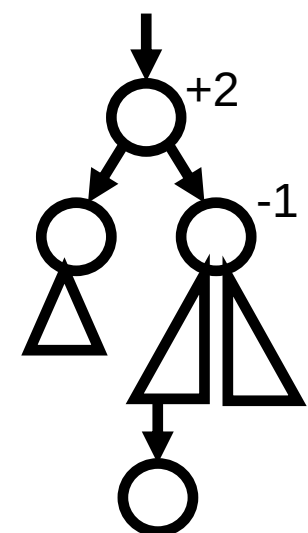
Single left rotation



Single right rotation



Double left-right rotation

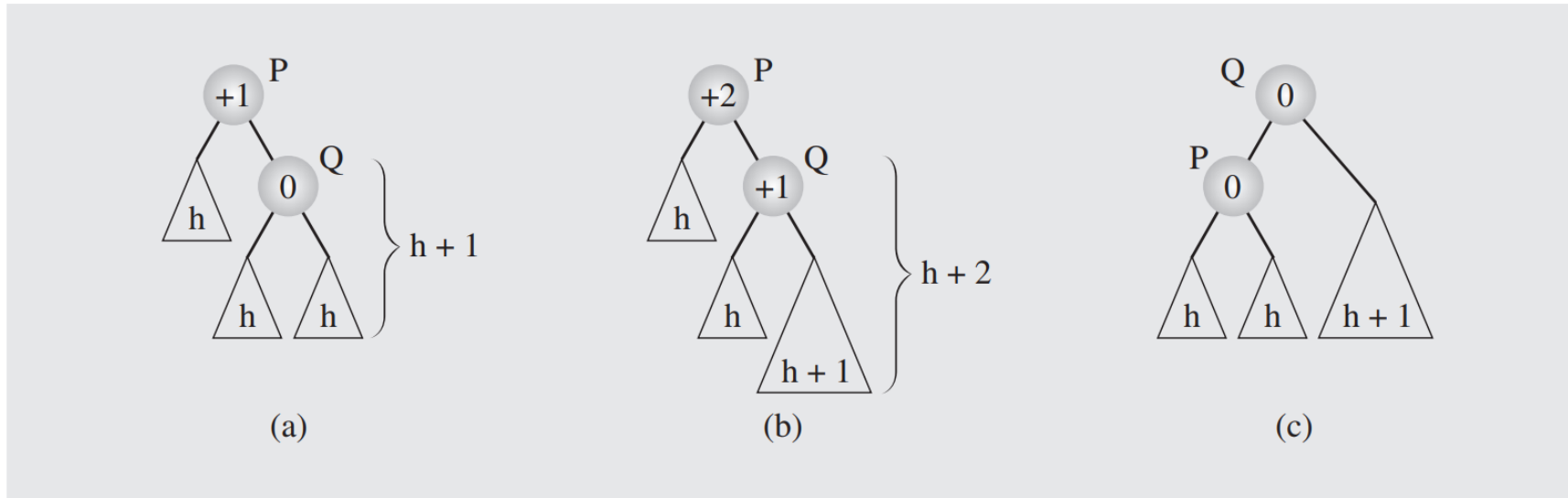


Double right-left rotation

Example of case 1(a)

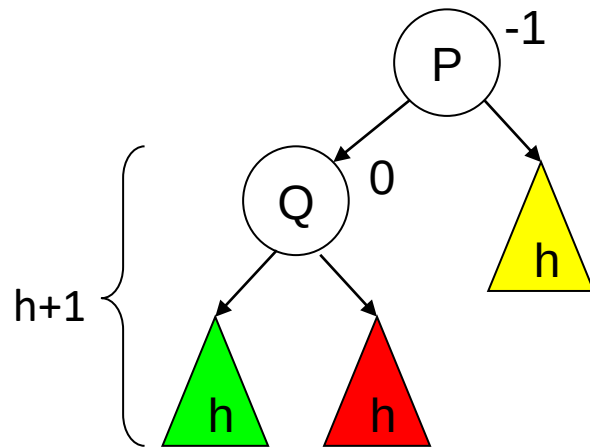
- The initial AVL tree is balanced (Fig. 6.41 (a)).
- A node is inserted to right subtree of Q causing imbalance of node P (Fig. 6.41 (b)).
- We do left rotation at P to make Q root and P left child of Q (Fig. 6.41 (c)).
- Left child of Q becomes right child of P.

FIGURE 6.41 Balancing a tree after insertion of a node in the right subtree of node Q.

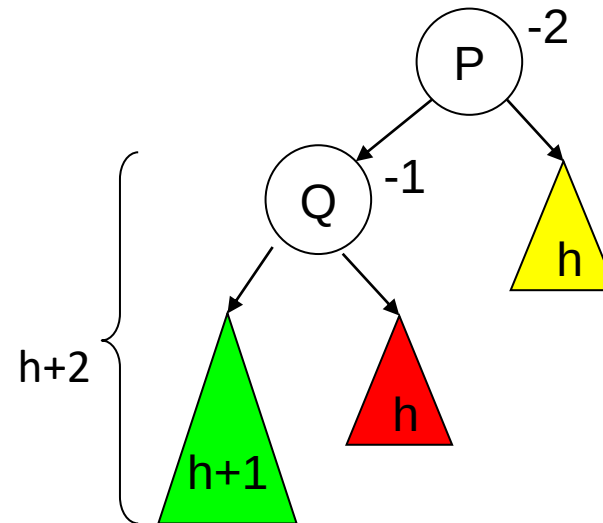


Example of case 1(b)

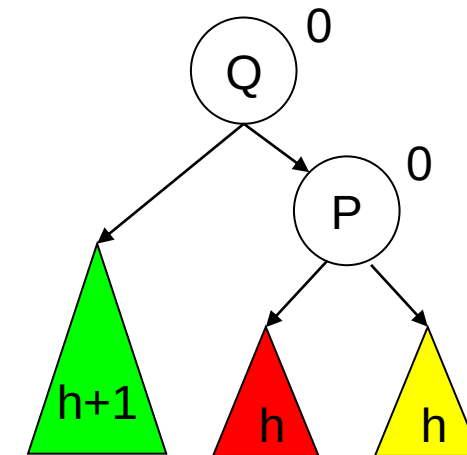
- The initial AVL tree is balanced (i).
- A node is inserted to left subtree of Q causing imbalance of node P (ii)
- We do right rotation at P to make Q root and P right child of Q (iii)
- Right child of Q becomes left child of P (iii)



(i)



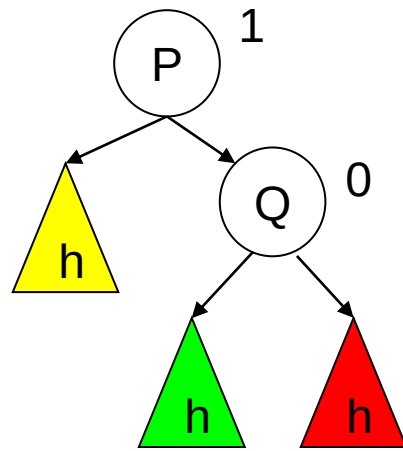
(ii)



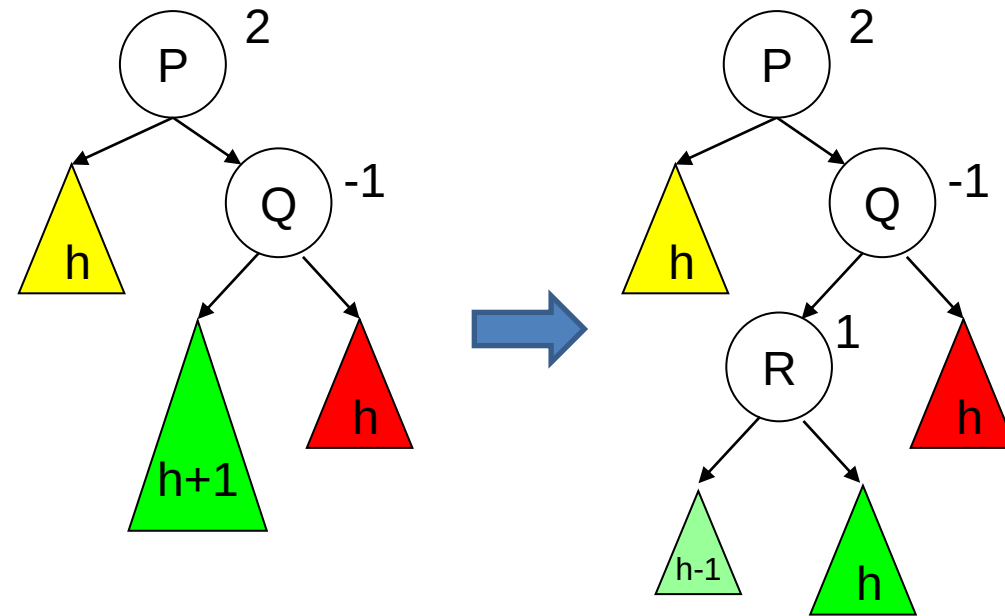
(iii)

Example of case 2 (a)

- We have a balanced AVL tree (i).
- A node R with a balance factor of +1 (or -1) is inserted to the left of Q causing imbalance at P (ii).



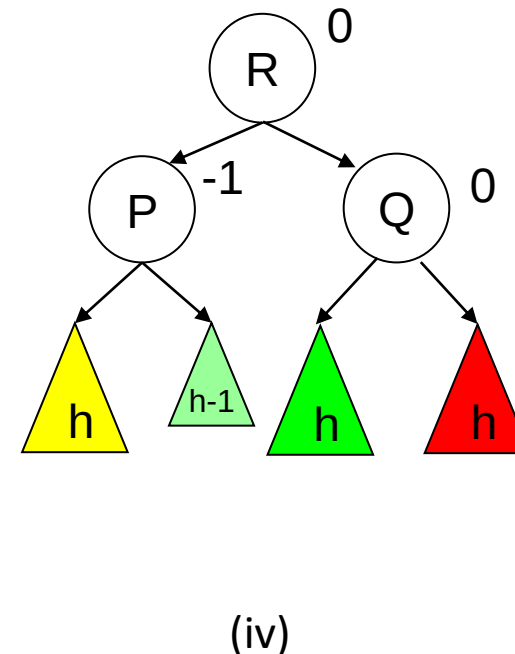
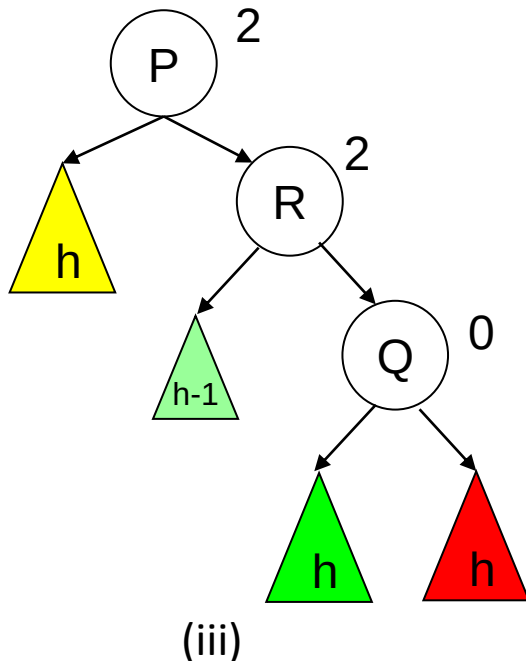
(i)



(ii)

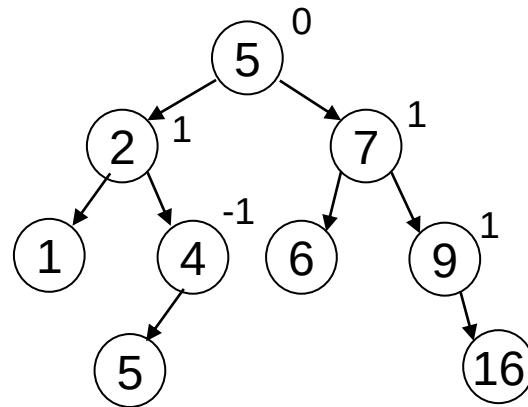
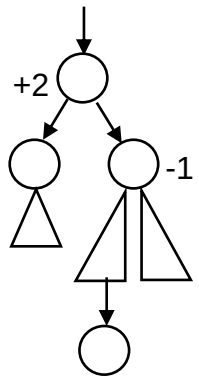
Example of case 2 (a)

- We do right rotation of R about Q to make R root and Q right child of R (iii).
 - Since R had a balance factor of 1, after right rotation it becomes 2 (iii).
- We now do left rotation at R to balance R (iv).
 - P is made left child of R, the left subtree of R becomes the right subtree of P (iv).



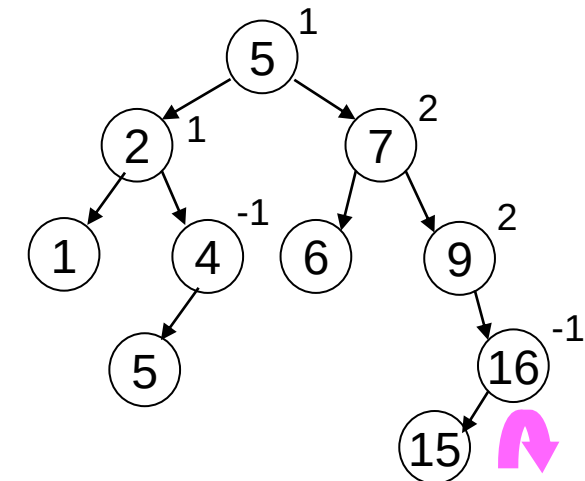
Another example of case 2 (b)

- Case 2b: The lowest node (with a balance factor of +2) had a taller **right-subtree** and the insertion was on the **left-subtree** of its right child.
- Requires a double right-left rotation to rebalance.



Original AVL Tree

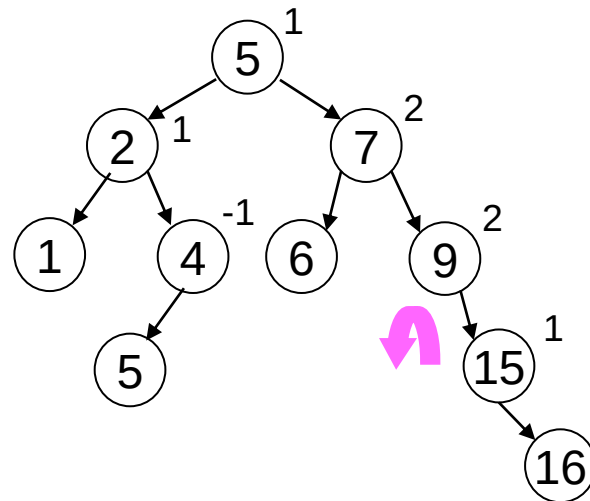
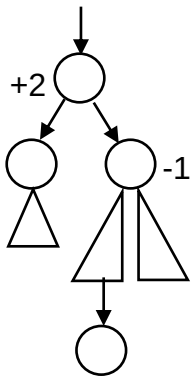
Insert 15



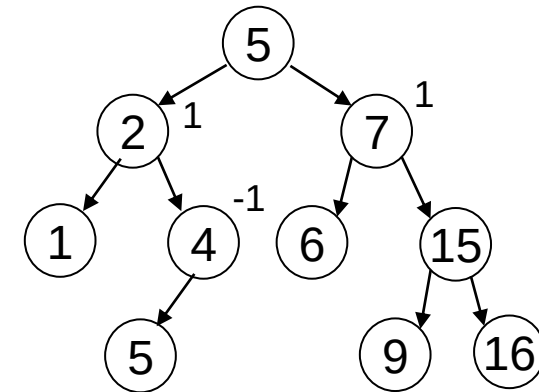
AVL Tree after insertion

Another example of case 2 (b)

- Case 2b: The lowest node (with a balance factor of +2) had a taller **right-subtree** and the insertion was on the **left-subtree** of its right child.
- Requires a double right-left rotation to rebalance.



AVL Tree after right
rotation with node 16 as
pivot



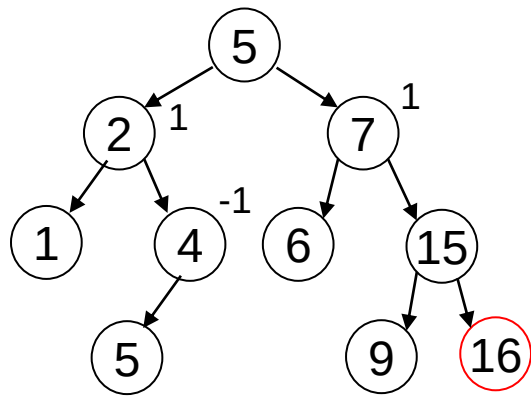
AVL Tree after left
rotation with node 9 as
pivot

Deletion

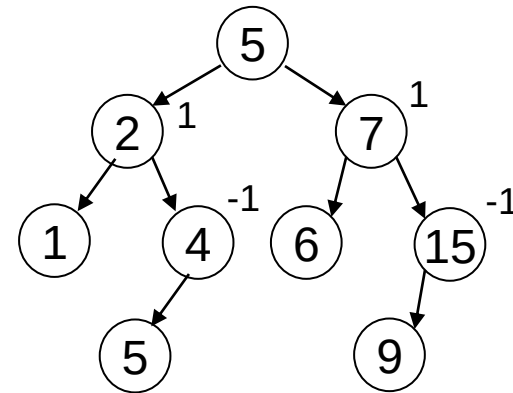
- Delete a BST node using deletion by copying algorithm.
- Rebalance the tree if an imbalance occurs.
- There are three deletion cases:
 1. Deletion that does not cause an imbalance.
 2. Deletion that requires a single rotation to rebalance.
 3. Deletion that requires two or more rotations to rebalance.

Deletion: Case 1 Example

- Case 1: Delete does not cause any imbalance
- Example delete node 16

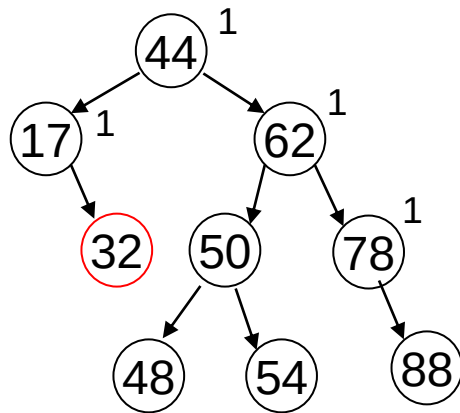


Delete 16

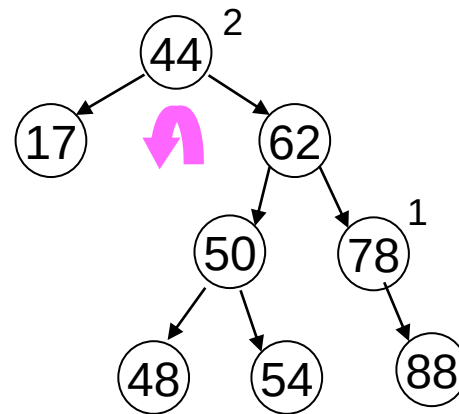


Deletion: Case 2 Example

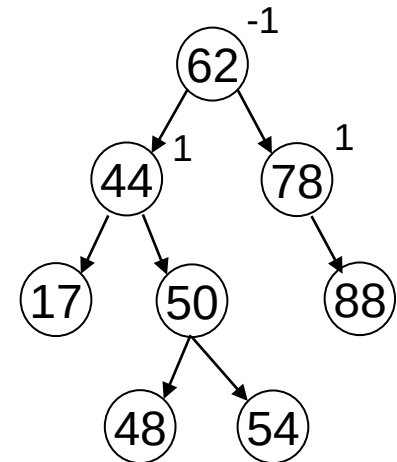
- Case 2: Deletion that requires a single rotation to rebalance.
- Example delete node 32



Delete node 32



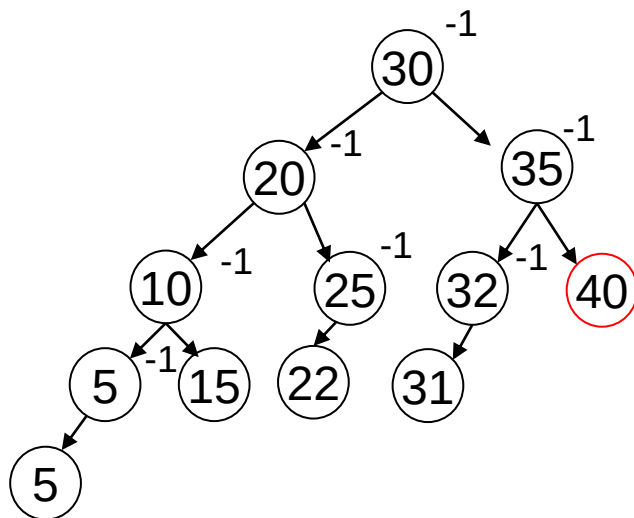
Left rotation with node 44 as the pivot



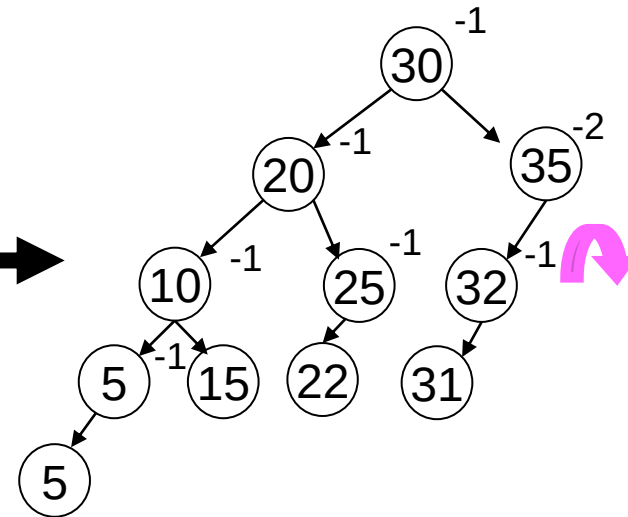
Final AVL Tree

Deletion: Case 3 Examples

- Case 3: Deletion that requires two or more rotations to rebalance.
- Example delete node 40



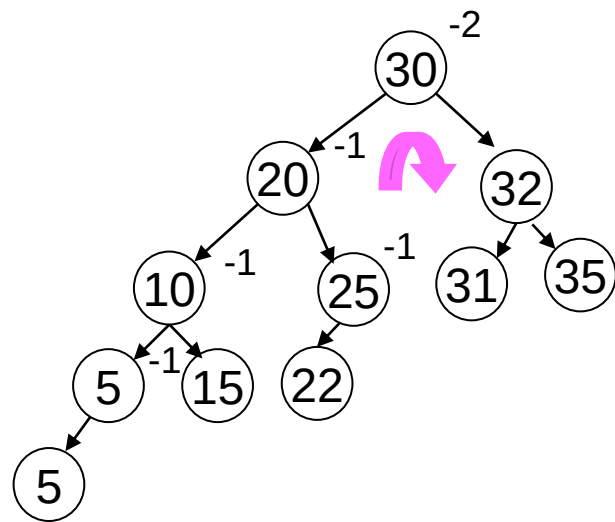
Delete Node 40



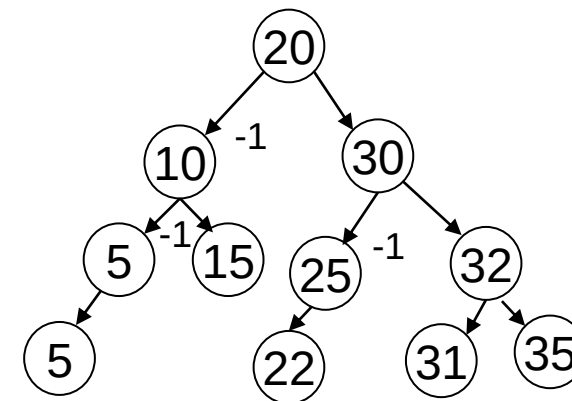
right rotation, with node 35
as the pivot

Deletion: Case 3 Examples

- Case 3: Deletion that requires two or more rotations to rebalance.
- Example delete node 40



right rotation, with node 30
as the pivot



Final AVL Tree



Book Reading

- Goodrich
 - AVL Trees, Chapter 11 section 11.3, pages: 481-489
- Adam Drozdek
 - AVL Trees, Chapter 6, section 6.7.2, pages: 256-261



Next Lecture

- Lecture 11